

Food Store — TP3

Optimización asistida por IA: filtros, planes de ejecución e índices

Base de Datos II — UTN · Equipo G · 22/09/2026

Esquema + carga masiva (Parte 1) · Table comparativa antes/después (Parte 2)

Lectura crítica de planes interpretados por IA (Parte 3) · Equivalencia de consultas con EXCEPT (Parte 4)

Desafío de la competencia: Q6 ×42 (Parte 5) · DUIA

Informe de laboratorio — Optimización con EXPLAIN ANALYZE (TP3, Parte 2)

Materia: Base de Datos II — UTN | **Proyecto:** Food Store **Base:** `food_store_tp3` (copia, sin triggers del TP2) · **Volumen:** 50.006 productos, 20.003 clientes, 200.003 pedidos, 500.014 líneas

Metodología (flujo profesional de la cátedra)

1. Medir antes con `EXPLAIN (ANALYZE, BUFFERS)` **en caliente** (2.^a corrida) y guardar el plan completo en `docs/planes/*_antes.txt`.
2. Pasar el plan real a la IA (OpenCode) → propone índices/reescrituras justificando cada uno por el nodo que ataca.
3. Leer cada propuesta línea por línea; solo se acepta lo que se puede explicar frente al plan.
4. Aplicar sobre la copia, `ANALYZE`, volver a medir → `docs/planes/*_despues.txt`.
5. Completar la tabla comparativa. **Lo que no mejora se documenta, no se silencia.**

2.2 Tabla de resultados (Execution Time real, ms, en caliente)

Consulta	Plan antes (nodo, cost est., t. real)	Cambio aplicado	Plan después (nodo, cost est., t. real)	Mejora
Q6_competencia (productos cat. 2, precio 600–3000, activos, orden por precio)	Bitmap Index Scan <code>idx_producto_categoria</code> (col. única) + Bitmap Heap recheck de precio + Sort. Cost 0.00..289.41; 14.242 ms	OPT-1 <code>CREATE INDEX ix_producto_cat_precio ON producto (categoria_id, precio) WHERE activo</code>	Index Scan <code>ix_producto_cat_precio</code> ; el rango pasa a <code>Index Cond</code> , pre-sorted por precio. Cost 1.33..44.40; 0.338 ms	×42 ✓
Q1_carta (Pizzas, precio 700–4500, orden por nombre)	Bitmap en <code>idx_producto_categoria</code> (16.669 filas) + Heap con recheck precio (Rows Removed by Filter 2.577) + Sort nombre. 30.645 ms	OPT-1 (mismo índice; esperado: atacar el recheck). El optimizador no lo eligió : siguió con el bitmap + recheck (plan casi idéntico). 25.703 ms	No sustancial (≈1,19×) — el índice nuevo sale más caro para este plan y el Sort por <code>nombre</code> domina. Se documenta.	
Q3_top_productos (top 10 por cantidad)	Seq Scan <code>detalle_pedido</code> (500.014 filas) + Hash Join + HashAggregate (Group Key p.id, 3.865 kB) + Sort top-N. 553.600 ms	OPT-2 covering <code>(producto_id) INCLUDE (cantidad)</code> (Index-Only Scan)	El planner desestimó el covering (Seq Scan igual; el costo del agregado domina). 508.222 ms	Marginal (≈1,09×) — el HashAggregate sobre 500k filas es el cuello real. Se documenta.
Q4_facturacion_clientes (suma por cliente, solo ENTREGADO)	Parallel Seq Scan <code>pedido</code> con Filter estado (Rows Removed by Filter ~49.977 c/worker, 50%) + HashAggregate en 5 batches con spill a disco (224 kB) + Gather. 431.486 ms	OPT-3 <code>CREATE INDEX ix_pedido_estado ON pedido (estado)</code> — hipótesis: recortar el scan de pedido	El nodo dominante era el agregado/hash sobre 500k líneas, no el filtro de estado: el plan no cambió. 491.502 ms	No mejora — la hipótesis era incorrecta; se descarta y documenta.

Q2_historial_cliente (0.398 ms) y Q5 no se tocaron: ya resolvían con índices existentes (uq_detalle_pedido_producto / idx_pedido_cliente).

Criterio de aceptación (aplicado)

- **Se aceptó** OPT-1 → explicable por el plan (el rango de precio se filtraba en el heap tras el bitmap; al llevarlo al `Index Cond` se acorta el recorrido y el orden queda pre-sorted). Verificado: ×42.
- **Se probó y documentó** OPT-2 y OPT-3 → se explicó la hipótesis, se midió el resultado real y se registró por qué no cambió el plan (no se descartaron en silencio).
- **Rechazo explícito:** ninguna propuesta se aplicó "porque la IA lo dijo"; cada una tenía un nodo concreto que justificar y una medición que la confirmara o rebatiera.

Planes completos (verbatim): `docs/planes/*_antes.txt` y `docs/planes/*_despues.txt`.

Ejercicio de lectura crítica: planes de ejecución interpretados por IA (TP3, Parte 3)

Materia: Base de Datos II — UTN · **Proyecto:** Food Store **Base:** `food_store_tp3` (50.006 productos, 200.003 pedidos, 500.014 líneas)

Contexto

Tras la Parte 2, el plan post-optimización de la consulta de la competencia (`Q6_competencia`, `docs/planes/Q6_competencia_despues.txt`) se le pasó a la IA para que lo **explicara en lenguaje natural**. El ejercicio consiste en confrontar esa explicación con el plan real: la IA es buena resumiendo, pero puede **plantar certezas falsas** (confundir cost estimado con tiempo, inventar claves de orden, sobrevender el efecto de un índice). Tabla de afirmaciones con evidencia textual del plan.

3.1 Explicación generada por la IA (textual, sobre Q6_competencia después)

"El plan usa un `CREATE INDEX` nuevo sobre `(categoria_id, precio)`. Con el índice la consulta ya no toca la tabla: hace un Index Scan que recorre la categoría 2 **entera** y descarta sobre la marcha los precios fuera del rango. Después un Incremental Sort **ordena las 8.882 filas** por nombre y el Limit deja las 100 primeras. El costo estimado es de 1.33 a 44.40, es decir que el plan **cuesta 44 milisegundos**; el Execution Time de 0.338 ms lo confirma. Como se leen exactamente las 100 filas del LIMIT, la mejora respecto de antes (14 ms) es de unos 42x, que es lo que esperábamos."

3.2 Análisis crítico contra el plan real

#	Afirmación de la IA	¿Correcta?	Corrección · evidencia textual del plan
1	"El Incremental Sort ordena las 8.882 filas por nombre "	X	La clave de orden es precio, id , no nombre: <code>Sort Key: precio, id</code> . Además 8.882 es la estimación del optimizador, no lo ordenado: <code>Full-sort Groups: 4 · Sort Method: quicksort</code> y solo se leyeron 103 filas .
2	"Recorre la categoría 2 entera y filtra sobre la marcha"	X	El rango de precio está en el <code>Index Cond: ((categoria_id = 2) AND (precio >= '600') AND (precio <= '3000'))</code> . El árbol del índice salta directo al rango (<code>Index Searches: 1</code>), no recorre todos los productos de la categoría.
3	"El cost estimado (44.40) son 44 milisegundos "	X	El <code>cost</code> de EXPLAIN es una unidad adimensional (I/O + CPU ponderados), no tiempo. Execution Time de 0.338 ms no "confirma" un costo en ms; son magnitudes distintas.
4	"Se leen exactamente las 100 filas del LIMIT"	Parcial	El Index Scan produjo 103 filas (<code>rows=103.00</code>) y el Limit cortó a 100. El Incremental Sort lee filas extra del grupo para garantizar el orden determinístico <code>precio, id</code> .
5	"La mejora 14 ms → 0.338 ms es ~42x"	✓	Correcto: 14.242 ms (antes, bitmap) → 0.338 ms (después, index scan). El único dato verificable de la explicación.

Conclusión de la lectura crítica: 4 de 5 afirmaciones eran parcial o totalmente incorrectas; solo la cuenta de mejora era cierta. La lección para el informe: **nunca citar la explicación de la IA como si fuera el plan** — toda afirmación se cita con su nodo/valor textual de `EXPLAIN (ANALYZE, BUFFERS)`.

3.3 Segundo caso: refutación de una hipótesis de la IA (Q4, OPT-3)

La IA propuso `ix_pedido_estado` para Q4 con la hipótesis: "el filtro `estado = 'ENTREGADO'` descarta ~50% de pedidos en un Seq Scan; indexando el estado el scan se acorta". La lectura crítica del plan final lo refuta:

Hipótesis de la IA	Evidencia del plan real que la refuta
"El cuello es el filtro de estado sobre pedido"	El plan muestra que el costo dominante es el agregado sobre 500.014 líneas: <code>HashAggregate</code> en <code>Batches: 5</code> con spill a disco (<code>Disk Usage: 224kB</code>), Hash Join sobre <code>detalle_pedido</code> (Parallel Seq Scan, <code>rows=166671</code> por worker) y Gather. El filtro de pedido (<code>Rows Removed by Filter: 49977</code> c/worker) es un costo menor comparado.
"El índice de estado va a recortar la medición"	Medido tras aplicar OPT-3: 431.486 ms → 491.502 ms (sin cambio, dentro del ruido). El plan no eligió el índice nuevo ni podía: el agregado de 500k filas no se reduce con un índice de estado.

Regla aplicada: la hipótesis se escribió, se probó en el motor y se descartó/documentó. No se dejó la afirmación de la IA como conclusión.

Planes usados: `docs/planes/Q6_competencia_despues.txt`, `docs/planes/Q4_facturacion_clientes_antes.txt`, `docs/planes/Q4_facturacion_clientes_despues.txt`.

Informe de equivalencia de consultas (TP3, Parte 4)

Materia: Base de Datos II — UTN · **Proyecto:** Food Store **Base:** `food_store_tp3` · **Script:** `sql/equivalencias_consultas.sql`

Dos requisitos en lenguaje natural. Cada uno se entregó como SQL generado por IA (versión A) y una versión alternativa de la **misma** semántica (versión B). La equivalencia se comprobó con `EXCEPT` en **ambas direcciones**: si `(A EXCEPT B)` y `(B EXCEPT A)` devuelven 0 filas, los conjuntos son iguales.

Espec 1 — resumen/agregación

Listar cada categoría vigente con la cantidad de productos vigentes que tiene, incluyendo las categorías sin productos vigentes (0), ordenado de mayor a menor cantidad.

- **A (IA):** `LEFT JOIN` sobre `producto` filtrado por `activo` + `COUNT(p.id)`.
- **B (alternativa):** subconsulta escalar `(SELECT COUNT(*) ...)`.
- Resultado esperado: 3 filas (3 categorías vigentes, 1 con 0 productos).
- Verificación: `A=B=3 filas` · `A-B = 0` · `B-A = 0` ✓

Espec 2 — comparación con un agregado global

Clientes que hayan facturado más que el promedio facturado por cliente, considerando solo pedidos ENTREGADO. Mostrar id, apellido, nombre y total facturado, de mayor a menor.

- **A (IA):** `GROUP BY` + `HAVING SUM(...) > (SELECT AVG(f) FROM (...))`.
- **B (alternativa):** CTE de totales + CTE del promedio + comparación.
- Resultado esperado: coincidir en número de filas y contenido.
- Verificación: `A=B=9.088 filas` · `A-B = 0` · `B-A = 0` ✓

Errores que la verificación EXCEPT destapó

1. **Correlación accidental:** en la subconsulta de A (y en la del HAVING) el alias del detalle interno quedó como `d` (externo) en vez de `d2`; la query corría pero correlacionaba contra el detalle del cliente externo, rompiendo la semántica (A daba 0 filas vs 9.088 de B). El `EXCEPT` con dirección detectó la discrepancia; se corrigió a `d2` y quedó documentado en el script.
2. **Ámbito de CTE:** las consultas de verificación referenciaban los CTE `totales` y `prom`, que solo existen dentro de su propia sentencia → se reescribieron como subconsultas autocontenidas.

Otra equivalencia verificada (espuria, Parte 1): la consulta de la competencia sin índice y con índice no devuelve iguales filas (el índice no cambia el resultado: `SELECT ... EXCEPT SELECT ...` = 0 en ambas direcciones, no se compara acá porque EXCEPT ignora orden; la verificación de orden quedó a cargo del plan `ORDER BY precio, id`).

Registro del desafío de la competencia (TP3, Parte 5)

Equipo G · Proyecto: Food Store · Fecha: 22/09/2026 Herramienta de IA: OpenCode (CLI, modelo *big-pickle)

Consulta fija de la competencia

Consulta de negocio con filtros y ordenamiento sobre la copia `food_store_tp3` (50.006 productos activos, 3 categorías):

```
SELECT p.id, p.nombre, p.precio
FROM producto p
WHERE p.categoria_id = 2
      AND p.precio BETWEEN 600 AND 3000
      AND p.activo
ORDER BY p.precio, p.id
LIMIT 100;
```

Estrategia aplicada (filtros + índices)

- 1. Sin índice dedicado (plan base): `Bitmap Index Scan` sobre `idx_producto_categoria` (columna única por categoría) + `Bitmap Heap Scan` que **re-filtra** el rango de precio en el heap (~16.700 activos de la categoría 2) + `Sort` manual por `precio, id`.
- 2. Optimización propuesta por IA y aplicada (OPT-1): `CREATE INDEX ix_producto_cat_precio ON producto (categoria_id, precio) WHERE activo;` — el índice parcial resuelve el filtro de precio en el árbol (`Index Cond`) y deja el orden `precio, id` **presorted** (el `Incremental Sort` solo ordena 4 grupos).

Medición final (EXPLAIN (ANALYZE, BUFFERS), en caliente)

Fase	Nodo dominante	Cost estimado	Execution Time real
Antes (sin índice dedicado)	Bitmap Index Scan <code>idx_producto_categoria</code> + Heap recheck + Sort	0.00..289.41	14.242 ms
Después (con <code>ix_producto_cat_precio</code>)	Index Scan <code>ix_producto_cat_precio</code> (103 filas) + Incremental Sort	1.33..44.40	0.338 ms

Mejora: 14.242 / 0.338 = x42.

Detalle del plan ganador

```
Limit (cost=1.33..44.40 rows=100 width=32) (actual time=0.173..0.296 rows=100 loops=1)
-> Incremental Sort (cost=1.33..3827.37 ...) Sort Key: precio, id
    Presorted Key: precio · Full-sort Groups: 4
-> Index Scan using ix_producto_cat_precio on producto p
    Index Cond: ((categoria_id = 2) AND (precio >= '600') AND (precio <= '3000'))
```

Transparencia: qué se probó y se descartó

- `ix_pedido_estado` (Q4): la hipótesis de la IA fue refutada por el plan (el cuello era el agregado de 500k líneas, no el filtro de estado) — documentado en `docs/informe_optimizacion.md` §2.2 y `docs/ejercicio_lectura_critica_planes_ia.md` §3.3.
- Covering index en `detalle_pedido` (Q3): aplicado, pero el optimizador desestimó el Index-Only Scan (agregado dominante) — mejora marginal, documentada.

Plan completo: `docs/planes/Q6_competencia_antes.txt` y `docs/planes/Q6_competencia_despues.txt`.

DUIA — TP3: Optimización asistida por IA (Food Store)

Materia: Base de Datos II — UTN · **Proyecto:** Food Store · **Fecha:** 22/09/2026

Herramientas usadas

Herramienta	Uso
OpenCode (CLI de IA, modelo *big-pickle)	Propuesta de índices para los planes medidos, generación/reescritura de consultas y explicación de planes (después confrontada de forma crítica).
PostgreSQL 18 (local)	Motor donde se midieron los planes (<code>food_store_tp3</code> , 50k productos / 200k pedidos / 500k líneas) y se verificaron las equivalencias.
psql (CLI)	Ejecución de <code>EXPLAIN (ANALYZE, BUFFERS)</code> , <code>ANALYZE</code> , índices y consultas de verificación.
Python runner (<code>run_explain.py</code>)	Correr EXPLAIN sobre <code>sql/queries.sql</code> y archivar los planes en <code>docs/planes/</code> .
Git + gh	Versionado y publicación en el repositorio <code>rayssa-ale/food-store</code> .

Para qué se usó la IA y con qué prompts/specs

Uso	Prompt / especificación aplicados
Elección de consultas lentas	«Traé 4 consultas de negocio reales con JOIN/filtros/ORDER/GROUP sobre producto/pedido/detalle para medir con EXPLAIN ANALYZE (carta, historial por cliente, top productos, facturación y una consulta "a lo competencia" sin índice dedicado)» → <code>sql/queries.sql</code> .
Propuesta de optimización (parte 2)	Se pasaron los planes reales (nodo, cost, times) junto con el pedido «proponé índices/reescrituras, y justificá cada propuesta por el nodo exacto que ataca». Salida: OPT-1 (compuesto parcial), OPT-2 (covering), OPT-3 (estado).
Lectura crítica (parte 3)	«Explicá en lenguaje natural este plan EXPLAIN» → se comparó afirmación por afirmación con el plan (4 de 5 imprecisiones detectadas). El proceso de refutación de una hipótesis propia (OPT-3/Q4) también quedó registrado.
Generación de consultas equivalentes (parte 4)	«Dadas estas 2 specs en lenguaje natural, generá el SQL y una versión alternativa con la MISMA semántica». Se eligieron LEFT JOIN+COUNT vs subconsulta escalar, y HAVING+subconsulta vs CTE. Bugs de correlación (alias <code>d</code> vs <code>d2</code>) descubiertos por la verificación EXCEPT en el motor , no por la IA.
Desafío de la competencia (parte 5)	«Optimizá esta consulta fija sobre la copia» → índice compuesto parcial <code>(categoria_id, precio)</code> WHERE <code>activo</code> , verificado x42.

Verificación (regla del TP: la IA propone, el motor decide)

- Parte 1 (datos):** conteos reales en `food_store_tp3` (50.006 / 20.003 / 200.003 / 500.014) y `ANALYZE` ✓
- Parte 2 (optimización):** Q6_competencia 14.242 ms → **0.338 ms (x42)** ✓ · Q1 ~1.19× sin cambio de plan ✓ documentada · Q3 ~1.09× marginal ✓ · Q4 sin mejora (hipótesis refutada) ✓ documentada — tabla en `docs/informe_optimizacion.md` .
- Parte 3 (lectura crítica):** cada afirmación de la IA contrastada con nodos textuales del plan (Sort Key, Index Cond, rows actuales, cost vs ms) ✓

4. **Parte 4 (equivalencia):** Espec 1 → 3 filas en ambas versiones; Espec 2 → 9.088 filas en ambas; `(A) EXCEPT (B) = 0` y `(B) EXCEPT (A) = 0` en ambos casos ✓ (script `sql/equivalencias_consultas.sql`).
5. **Parte 5 (competencia):** plan base vs plan optimizado con EXPLAIN ANALYZE, ×42, registro en `docs/registro_competencia.md` ✓

Revisión humana aplicada

- Cada propuesta de índice se leyó y justificó **antes** de aplicarse (qué nodo atacaba y por qué); ninguna se corrió "porque la IA lo dijo".
- Las mediciones provienen de corridas **en caliente** de `EXPLAIN (ANALYZE, BUFFERS)`, transcritas textuales a `docs/planes/*.txt`.
- Las consultas equivalentes se validaron en el motor con EXCEPT en ambas direcciones (0 filas), no por inspección visual.
- Trabajo sobre la **copia** `food_store_tp3` (`protocolo_seguridad.md`); los índices de optimización quedan en `sql/optimizaciones.sql`, no en `schema.sql`.